



WEST UNIVERSITY OF TIMIȘOARA
FACULTY OF COMPUTER SCIENCE
BACHELOR STUDY PROGRAM: ARTIFICIAL INTELLIGENCE

CRM – Customer Relationship Management

BACHELOR THESIS

SUPERVISOR:

Assist. Prof. Ph.D. Sebastian-Aurelian
Ștefăniță

GRADUATE:

Raul Veca

Timișoara 2026

Abstract

This thesis presents a CRM (Customer Relationship Management) core developed in Java for a company that sells IT training courses. The work is organized in two complementary parts. The first part establishes the theoretical foundations and the state of the art, surveying the definition and evolution of CRM systems, modern architectural paradigms, dominant commercial and open-source platforms, total cost of ownership, integration with AI and ML, security assurance, the impact of cloud computing and containerization, and emerging trends. The second part is a detailed technical dissection of the implemented system. Rather than a simple CRUD application, the project is designed as a software-architecture demonstration in which a business problem is decomposed into clearly separated layers and components that apply well-known design patterns. The domain naturally accommodates two customer categories — B2C individuals seeking training and B2B companies purchasing corporate programs — both unified under a single **Contact** entity. The system is built around a layered architecture (UI → Facade → Command → Service → Repository → DAO), an event-driven notification mechanism, and interchangeable scoring and pricing strategies, making explicit use of the Singleton, Facade, Command, Observer, Strategy, Builder, Factory, Chain of Responsibility, Template Method and Repository/DAO patterns. The codebase is intended as a production-ready core whose service and persistence logic could later be exposed through a REST API and web UI without changing the underlying business rules.

Statement of Scientific Integrity

I declare that this BSc thesis is an original report and represents my own work, conducted under the supervision of Assist. Prof. Ph.D. Sebastian-Aurelian Ștefăniță. All research methods, data collection, analysis and interpretation presented in this thesis are accurately reported, and all sources have been cited appropriately where I have made use of the ideas and/or visuals of others.

Contents

I	Theoretical Foundations and State of the Art	1
1	Introduction	2
1.1	About This Paper	2
1.2	Project Objectives	3
2	CRM Systems – State of the Art	4
2.1	Definition and Evolution of CRM Systems	4
2.2	Modern Architecture of CRM Systems	4
2.3	Dominant CRM Technologies and Platforms	5
2.3.1	Enterprise-Class Solutions (Tier 1)	6
2.3.2	The Open-Source Paradigm: Flexibility and Technological Sovereignty	6
2.3.3	Specialized and Best-of-Breed Solutions	6
2.4	Comparative Analysis of Total Cost of Ownership (TCO): SaaS vs. Open-Source	7
2.4.1	Cost Dynamics: OPEX vs. CAPEX	7
2.4.2	Vendor Lock-in Risk	7
2.4.3	Maintenance and Internal Expertise	8
2.4.4	Integration Costs	8
2.5	Integration with AI and ML	8
2.6	Current Challenges and Emerging Trends	9
2.6.1	The Omnichannel Paradigm and Customer Journey Mapping	9
2.6.2	Data Governance and Regulatory Compliance (GDPR and CCPA)	9
2.6.3	Convergence with the Internet of Things (IoT) and Proactive Services	10
2.7	Security Assurance in Modern CRM Systems	10
2.8	The Impact of Cloud Computing and Containerization	11
2.9	Technical Challenges	12
II	Architecture and Implementation	14
3	Introduction: What Is the CRM Training IT Project?	15
4	Overview of Technology Stack and Project Structure	16
4.1	Technology Stack	16
4.2	Logging and Persistence Infrastructure	16
4.3	Package Structure	16
5	Application Startup and System Initialization	18
5.1	Configuration and Database Initialization	18
5.2	Event Registration and Shutdown	18
6	Layered Architecture	19
6.1	User Interface and Facade Layers	19
6.2	Service, Repository and DAO Layers	19
6.3	Simplified Flow	19
7	Domain Model and Main Entities	20

7.1	Base Entity	20
7.2	Contact	20
7.3	Course and CourseSession	20
7.4	Enrollment, Opportunity and Activity	20
8	Contacts, Leads and Score Calculation.....	22
8.1	Contact Creation Flow	22
8.2	Validation and Uniqueness	22
8.3	Lead Scoring Strategies	22
9	EventBus, Activities and Automated Reactions.....	24
9.1	Activity Creation	24
9.2	Activity Completion and Score Updates	24
9.3	Benefits of the Event-Driven Design.....	24
10	B2B Opportunities and the Sales Pipeline.....	25
10.1	Opportunity Modeling	25
10.2	Pipeline Stages	25
10.3	Opportunity Services and Events	25
11	Courses, Sessions, Enrollments and Pricing Strategies	26
11.1	Educational Entities	26
11.2	Course Service.....	26
11.3	Enrollment Service and Pricing Strategy	26
12	Persistence: Repository, DAO and JDBC	27
12.1	Repository and DAO Separation	27
12.2	AbstractRepository and AbstractDao	27
12.3	Concrete DAOs.....	27
12.4	Transactions and Partial Failures	28
13	Design Patterns and Their Role in the Project	29
13.1	Creational and Structural Patterns.....	29
13.2	Behavioral Patterns	29
13.3	Summary of the Design Patterns.....	30
14	Validation, Exceptions and Logging	31
14.1	Validation Chain	31
14.2	Exception Hierarchy	31
14.3	Logging	31
14.4	Validation Gaps	31
15	Database, SQL Schema and Consistency Observations	33
15.1	Conflicting Schema Sources	33
15.2	Schema Initialization	33
15.3	Schema Richness and Indexing.....	33
15.4	Auditing	34
16	Features and Non-Functional Requirements Derived from the Code	35
16.1	Functional Requirements	35
16.2	Non-Functional Requirements.....	35
16.3	Security	35
16.4	Scalability	35
17	Limitations and Possible Improvements.....	37
17.1	Key Limitations	37
17.2	Further Improvements	37

Bibliography 38

Part I

Theoretical Foundations and State of
the Art

Chapter 1

Introduction

In the current economic and technological landscape, defined by accelerated digitalization, CRM (Customer Relationship Management) platforms have become fundamental pillars of business strategies. These systems have surpassed the stage of simple contact management, evolving into complex business intelligence solutions that integrate operational workflows, predictive analytics, and automated decision-making systems. The preference for scalable, customizable solutions that are independent of major vendors has stimulated the rise of open-source alternatives, which guarantee full control over the codebase and enhanced adaptability to pre-existing digital infrastructures.

The present study conducts a thorough examination of a modern CRM system, developed on the Django Admin architecture and structured according to modular design principles. The research investigates essential parameters such as the configuration of internal components, design logic, integration of collaboration and analytics modules, database performance, as well as critical aspects related to security, extensibility, and legal compliance.

By highlighting both the benefits and the identified limitations, this paper aims to provide an objective perspective on the effectiveness of such a solution in the digital transformation process of a company. Additionally, future technological trends and the strategic relevance of adopting an open-source CRM in a competitive environment undergoing continuous reconfiguration are explored.

1.1 About This Paper

This paper carries out a technical dissection of an advanced CRM ecosystem, developed in the Java programming language, with the purpose of validating the reliability and performance of open-source architectures in volatile enterprise environments. The study investigates the modular decoupling of the system, which orchestrates functional entities (Tasks, CRM Core) with outbound marketing modules (Mass Mail) and messaging components (Chat), demonstrating a high degree of granularity and enhanced agility in deployment processes.

The architecture is notable for its implementation of predictive engines based on machine learning for anticipating consumer behavior and automated lead scoring, complemented by data stream processing mechanisms for funnel analysis and the automatic transformation of interactions into atomic tasks. The relational database schema is optimized for intensive transactional workloads and data persistence, ensuring high throughput and scalability in business scenarios of significant complexity.

Additionally, security vectors, hardening protocols, granular auditing mechanisms, and compliance with data protection standards are evaluated, with identified limitations including computational overhead and dependencies on the specific technology stack. The development roadmap targets the optimization of interoperability through extensible APIs, the integration of emerging technologies, and the consolidation of cross-platform components for improved synchronous collaboration.

1.2 Project Objectives

1. **Evaluation of the CRM system architecture** – The modular structure and design principles used for developing a scalable and adaptable platform are analyzed.
2. **Identification of competitive advantages of the open-source solution** – The benefits related to flexibility, reduced costs, and technological independence are highlighted.
3. **Analysis of analytical and predictive functionalities** – The manner in which artificial intelligence-based components contribute to strategic decision-making is detailed.
4. **Study of database performance and the relational model** – The architectural optimizations that enable scalability and execution speed are examined.
5. **Evaluation of security and compliance aspects** – The measures implemented for personal data protection and action traceability assurance are discussed.
6. **Identification of technical and organizational challenges** – A realistic picture of the difficulties encountered in adopting and maintaining such a system is provided.
7. **Outlining future development directions** – Solutions and innovations that can consolidate the CRM system's position in a continuously evolving digital environment are proposed.

Chapter 2

CRM Systems – State of the Art

2.1 Definition and Evolution of CRM Systems

Customer Relationship Management (CRM) [1, 2] constitutes a fundamental pillar of contemporary business strategies, facilitating the rigorous administration of interactions with current and potential customer segments. Defined as a philosophy centered on the end beneficiary and supported by complex information ecosystems, CRM aims to streamline commercial partnerships, increase loyalty, and, implicitly, enhance the financial performance indicators of the economic entity.

The evolutionary trajectory of the CRM concept is marked by profound structural transformations. In the 1980s and 1990s, the rudiments of this field materialized through Sales Force Automation (SFA) systems [3], primarily focused on contact tracking and opportunity management. These early solutions sought to streamline sales workflows, reduce repetitive administrative tasks, and ensure superior visibility over the commercial pipeline.

With the onset of the 2000s, CRM entered a maturation phase, incorporating extended marketing and post-sales support functionalities. Thus, the platforms surpassed their limited role as sales support tools, becoming integrated architectures designed to monitor the entire consumer lifecycle. This paradigm enabled the development of a holistic vision of multi-point interactions, consolidating the strategic bond between the organization and the client.

The 2010–2020 decade was defined by the proliferation of cloud computing technologies and the emergence of Social CRM [7]. The transition to cloud infrastructures democratized access to high-performance tools, minimizing the financial barriers of implementation and providing enhanced scalability. Simultaneously, the convergence with social networks facilitated the extraction of valuable datasets regarding the digital behavior of users, supplementing the informational profile stored in traditional systems.

At present, the CRM domain is undergoing a reconfiguration phase under the influence of Artificial Intelligence (AI), Machine Learning (ML) algorithms, and predictive analytics [11]. The integration of these advanced technologies enables hyper-personalization of the user experience and a superior capacity for anticipating consumption needs, offering a decisive competitive advantage to organizations that adopt these digital innovations.

2.2 Modern Architecture of CRM Systems

Contemporary CRM systems are grounded in advanced architectural paradigms, specifically designed to meet the demands of volatility and complexity inherent in today’s digital ecosystem. Surpassing the limitations of traditional monolithic structures, modern solutions adopt a modular and highly scalable approach, supported by cloud computing technologies and microservices-based architectures. This design strategy ensures optimal functional decoupling, allowing organizations enhanced agility in implementing new business requirements and facilitating the organic integration of technological innovations without compromising the stability

of the entire system.

Presentation Layer (Frontend and User Interface) The presentation layer represents the critical mediation interface between the system’s computational logic and the end user, playing a determinant role in the solution’s adoption rate at the organizational level. In next-generation CRM architectures, this layer is not limited to simple information display but delivers an omnichannel experience through Responsive Design web interfaces, native mobile applications, and Progressive Web App (PWA) solutions.

An essential aspect is the integration of interactive dashboards and real-time data visualization tools, which synthesize massive volumes of information into intuitive visual indicators. From a technological standpoint, the development of this layer relies on declarative frameworks and performant JavaScript libraries such as React, Angular, or Vue.js, which enable the creation of dynamic Single Page Application (SPA) interfaces. For the mobility component, cross-platform development solutions such as React Native or Flutter are frequently adopted, thus ensuring visual and functional consistency across different operating systems [13].

Business Logic Layer Considered the functional nucleus of the entire architecture, the business logic layer (or Application Layer) is where the processes, algorithms, and operational rules specific to the business model are orchestrated. It encompasses complex workflow engines capable of automating repetitive work processes, intelligent notification systems, and triggers for proactive alerts.

A vital component of this layer is represented by the analytical modules designed for real-time computation of Key Performance Indicators (KPIs). Furthermore, the system’s interoperability with other enterprise solutions (such as ERP or e-commerce platforms) is guaranteed through the exposure of API endpoints (RESTful or GraphQL). This approach facilitates a Headless architecture, granting developers the freedom to extend and customize CRM functionalities according to the specific business vertical in which the organization operates.

Data Persistence and Management Layer Data management in a modern CRM represents a major technological challenge, requiring a fine balance between integrity, availability, and performance [12]. The architecture of this layer frequently adopts the Polyglot Persistence model, employing Relational Database Management Systems (RDBMS) such as PostgreSQL or MySQL for storing transactional data that requires strict consistency (ACID).

Simultaneously, for managing unstructured data, interaction logs, or Big Data volumes, NoSQL databases (such as MongoDB, Cassandra, or Redis) are integrated, offering superior horizontal scalability. For the analytical dimension and historical reporting, the system can interface with Data Warehousing solutions, providing decision support based on aggregated data. The security of this layer is reinforced through rigorous encryption protocols at rest and in transit, automated backup mechanisms, and Disaster Recovery strategies, thus guaranteeing the organization’s informational resilience against any technical incidents.

2.3 Dominant CRM Technologies and Platforms

The contemporary technological landscape of Customer Relationship Management systems is defined by a pronounced taxonomic diversification, structured according to the delivery model, architectural complexity, and functional specificity. This market fragmentation allows

organizations to adopt solutions precisely calibrated to their operational scale and internal process maturity [10]. At present, three dominant strategic segments can be identified.

2.3.1 Enterprise-Class Solutions (Tier 1)

These platforms are designed to support massive data volumes and cross-departmental business processes, offering an exhaustive portfolio of integrated modules.

- **Salesforce:** Established as market leader through its multi-tenant cloud paradigm, offering a PaaS (Platform as a Service) ecosystem. The use of the proprietary Apex programming language and the Lightning Framework component development framework enables virtually unlimited extensibility, transforming the CRM into an operating system for the entire business.
- **Microsoft Dynamics 365:** Represents the strategic choice for organizations dependent on the Microsoft ecosystem. Its competitive advantage resides in native interoperability with the Microsoft 365 suite, the Power BI analytics platform, and the Azure infrastructure. This synergy facilitates a reduced learning curve and data consistency across the entire IT infrastructure [6].
- **Oracle CX Cloud:** Emphasizes high-performance transactional processing and deep integration with ERP systems, being optimized for sectors with strict regulations and complex data structures.

2.3.2 The Open-Source Paradigm: Flexibility and Technological Sovereignty

Open-source solutions have experienced a remarkable ascent, being preferred by organizations that prioritize full control over the infrastructure and the elimination of vendor lock-in.

- **SuiteCRM and EspoCRM:** These platforms offer a viable alternative to costly commercial solutions, allowing deep customizations at the core and database schema levels.
- **Odoo:** Distinguished by a unique modular architecture where the CRM is merely a component of a broader ERP ecosystem. This approach eliminates the need for complex third-party integrations, ensuring a fluid flow of information between the sales, inventory, and accounting departments.
- **OroCRM:** Specialized in the B2B and e-commerce sectors, offering advanced segmentation and purchasing behavior analysis tools, while benefiting from the robustness of the Symfony framework.

2.3.3 Specialized and Best-of-Breed Solutions

These platforms aim for excellence within a specific functional niche, often adopting a philosophy centered on user experience (UX) and rapid deployment.

- **HubSpot:** Revolutionized the market through the concept of Inbound Marketing, integrating content generation, SEO, and marketing automation tools into an extremely intuitive interface, optimized for the accelerated scaling of SMEs.
- **Pipedrive:** Focuses exclusively on sales process efficiency (Sales Pipeline Management). Its design philosophy emphasizes transaction flow visualization and activity-based selling, reducing administrative complexity for sales representatives.

- **Zendesk:** Although it has evolved into a full CRM, it remains the point of reference for Customer Service and Ticketing. Its architecture is optimized for omnichannel support, enabling the unified management of interactions originating from email, phone, chat, and social media.

2.4 Comparative Analysis of Total Cost of Ownership (TCO): SaaS vs. Open-Source

The choice of a CRM platform must not be based exclusively on technical functionalities but also on a rigorous evaluation of the Total Cost of Ownership (TCO). This represents the sum of all direct and indirect costs generated throughout the entire system lifecycle (acquisition, implementation, operation, maintenance, and decommissioning).

2.4.1 Cost Dynamics: OPEX vs. CAPEX

In the SaaS model, costs are predominantly of the OPEX (Operational Expenditure) type. The organization pays for the right of use, which provides short-term financial predictability but can lead to immense cumulative costs as the number of users grows. In contrast, the implementation of an open-source CRM may involve a higher initial CAPEX (Capital Expenditure) for configuration, security auditing, and training; however, the marginal cost per additional user tends toward zero, offering superior economic efficiency in the long term.

Evaluation Criterion	SaaS Model (e.g., Salesforce, Dynamics)	Open-Source Model (e.g., SuiteCRM, Odoo)
Licensing	Recurring costs (monthly/annual subscription per user).	Zero or negligible licensing costs.
IT Infrastructure	Included in the subscription price (vendor's cloud).	Requires own servers or private hosting (private cloud).
Deployment & Setup	Fast, based on predefined configurations.	Slower; involves manual installation, configuration and hardening.
Customization	Limited by the vendor's ecosystem and costly through APIs.	Total; unlimited access to source code for structural changes.
Maintenance & Updates	Managed automatically by the vendor.	Managed internally or via subcontractors (requires technical expertise).
Data Sovereignty	Data stored on the vendor's servers (compliance risks).	Absolute control over the location and security of the databases.
Scalability	Instant scaling by upgrading the pricing plan.	Technical scaling (vertical/horizontal) that requires hardware resources.

Table 2.1: Comparative Table: Cost and Resource Structure

2.4.2 Vendor Lock-in Risk

A critical factor in TCO analysis is the strategic dependency on the vendor. Proprietary solutions often impose high technical and financial barriers when the organization wishes to

migrate data to another platform. Open-source solutions eliminate this risk, offering technological freedom; the source code and database remain in the full ownership of the organization, allowing system adaptation regardless of the commercial policies of an external vendor.

2.4.3 Maintenance and Internal Expertise

Although open-source licensing is free of charge, the TCO includes the salaries of specialists (system administrators, Python/PHP developers, security experts) who ensure the platform's operation. In the SaaS case, these costs are externalized to the vendor; however, the organization loses the capacity to develop internal technical competencies specific to its own product.

2.4.4 Integration Costs

CRM systems do not operate in isolation. While enterprise platforms (SaaS) offer native connectors (“out-of-the-box”) for popular applications, integrating legacy or highly specific systems can become financially prohibitive due to the limitations of commercial APIs. The open-source model facilitates deep interoperability, enabling the development of custom connectors directly at the database or business logic level.

The SaaS model is optimal for organizations seeking rapid deployment and lacking a mature IT infrastructure. Nevertheless, for companies oriented toward strategic digitalization that require full control over data and complex customizations at reduced marginal costs, the open-source solution (such as the one based on Django Admin) represents a more sustainable investment.

2.5 Integration with AI and ML

The integration of Artificial Intelligence (AI) and Machine Learning (ML) algorithms into contemporary CRM architectures does not represent merely an incremental evolution but rather a vector of radical transformation in how organizations process their informational capital. This technological synergy enables the transition from reactive record-keeping systems to proactive decision-support platforms, capable of extracting non-linear correlations from massive and heterogeneous datasets.

The implementation of predictive analytics allows organizations to surpass simple historical reporting, offering advanced capabilities for forecasting and modeling consumer behavior [9].

- **Attrition Risk Modeling (Churn Prediction):** By training classification models (such as Random Forest or XGBoost) on historical transaction and interaction data, the system can detect early signals of customer loyalty erosion. This early identification enables the automatic triggering of personalized retention workflows.
- **Sales Forecasting and Lead Scoring:** ML algorithms evaluate the conversion probability of each commercial opportunity (Lead Scoring), allowing the sales force to prioritize efforts toward segments with the highest yield potential.
- **Dynamic Pricing Strategies:** The use of advanced regressions and Reinforcement Learning facilitates real-time price adjustments based on demand, inventory, and competitive behavior, thereby maximizing the profit margin.

CRM systems equipped with artificial intelligence components redefine the concept of customer experience through the generation of adaptive content.

- **Recommendation Engines:** Based on Collaborative Filtering or Content-based Filtering techniques, these modules analyze individual preferences and the consumption patterns of similar segments to propose products or services with maximum relevance [4].
- **Contextual Marketing Automation:** AI facilitates the delivery of targeted messages at the optimal moments within the acquisition cycle, significantly increasing click-through rates (CTR) and diminishing the informational noise perceived by the client.

The qualitative dimension of customer relations is enhanced by Natural Language Processing (NLP) technologies, which transform unstructured data into actionable information [8, 9].

- **Sentiment Analysis:** Through the use of Transformer-based models (such as BERT or derived architectures), the CRM can process feedback from emails, reviews, or social media to evaluate the emotional state of the customer base.
- **Chatbot and Virtual Agent Systems:** The integration of Generative AI and Large Language Models (LLMs) enables the development of conversational interfaces capable of resolving complex support requests, automatically classifying support tickets, and extracting key entities from dialogues, thereby reducing operational pressure on contact centers.

The adoption of these technologies transforms the CRM from a passive database into an autonomous growth engine, offering a critical competitive advantage through the reduction of informational asymmetry and the increase in market response speed.

2.6 Current Challenges and Emerging Trends

In the current landscape of digital transformation, the evolution of CRM systems is marked by three fundamental vectors: the need for a unified consumer experience, the rigor of legislative frameworks regarding privacy protection, and the interconnectivity offered by smart devices. These factors redefine software solution architecture from simple databases to intelligent relationship management ecosystems [5].

2.6.1 The Omnichannel Paradigm and Customer Journey Mapping

In a business environment characterized by the fragmentation of communication channels, organizations tend toward an omnichannel approach, surpassing the limitations of the traditional multi-channel strategy. While multi-channel assumes presence on multiple platforms (web, mobile, social media, email, physical points), omnichannel aims for their synergistic integration.

Modern CRM systems are designed to centralize heterogeneous information flows to provide a holistic (360-degree) view of the customer. This consolidation enables granular monitoring of each stage of the customer journey, facilitating contextual personalization that maximizes Customer Lifetime Value (CLV). From a technical standpoint, this involves the use of advanced data synchronization mechanisms and digital identity reconciliation, ensuring a fluid user transition between the virtual and physical environments.

2.6.2 Data Governance and Regulatory Compliance (GDPR and CCPA)

Information security and legal compliance have become central pillars in the design of contemporary CRMs. The implementation of rigorous regulations such as the General Data

Protection Regulation (GDPR) in the European space or the California Consumer Privacy Act (CCPA) in the United States has imposed the adoption of the Privacy by Design principle.

This transition generates significant architectural challenges, as systems must integrate robust modules for:

Consent Management: Explicit traceability of the user's agreement for each type of processing.

Right to Portability and Erasure: Automation of the processes for extracting or deleting personal data upon request.

Auditability and Transparency: Maintenance of detailed activity logs (audit logs) that allow the reconstruction of any data operation in the event of a regulatory audit.

Failure in technical alignment with these standards attracts not only drastic financial sanctions but also an irremediable erosion of the brand's trust capital.

2.6.3 Convergence with the Internet of Things (IoT) and Proactive Services

The integration of Internet of Things (IoT) technologies into the CRM ecosystem marks the transition from transaction-based relationship management to one based on actual product usage. Connected devices become active sensors that transmit constant telemetry streams directly into the CRM platform.

This connectivity enables organizations to adopt a proactive service model:

- **Predictive Maintenance:** Identifying anomalies in product operation before the user detects a malfunction, automatically generating a support ticket or a technical visit.
- **Behavior-Based Personalization:** Adjusting commercial offers based on the client's actual mode of interaction with purchased goods.
- **Supply Chain Optimization:** Synchronizing field consumption data with inventory and logistics, ensuring superior operational efficiency.

The synergy among these trends transforms the CRM system into the organization's nerve center, where technology does not merely store data but orchestrates experiences, guarantees processing ethics, and anticipates the future needs of the market.

2.7 Security Assurance in Modern CRM Systems

In a digital ecosystem defined by the proliferation of cyber threats, security does not represent merely a peripheral functionality but the foundation upon which the integrity and availability of the CRM system are built. Given the extremely sensitive nature of the managed data (personal information, trade secrets, pricing strategies), the security architecture must adopt a Defense in Depth approach.

A robust strategy for the protection of informational assets encompasses the following critical dimensions:

1. **Access Control and Identity Management (IAM).** Rigorous access control constitutes the first line of defense against unauthorized intrusions. *Multi-Factor Authentication (MFA)*: The implementation of MFA adds an additional layer of verification, mitigating

the risks associated with credential compromise through phishing or brute-force techniques. *Single Sign-On (SSO)*: The use of SSO protocols (such as SAML or OpenID Connect) enables centralized identity management, reducing the attack surface and improving the user experience by eliminating credential set fragmentation.

2. **Data Protection through Advanced Encryption.** Ensuring data confidentiality requires the implementation of high encryption standards throughout the entire data lifecycle. *Data-in-Transit*: The use of latest-generation TLS (Transport Layer Security) protocols to secure client-server communications, preventing Man-in-the-Middle attacks. *Data-at-Rest*: Encryption of databases and stored files (using algorithms such as AES-256) guarantees that, in the event of unauthorized physical access to the storage medium, the information remains indecipherable.
3. **Authorization Model and the Principle of Least Privilege.** The software architecture must reflect the organizational structure through Role-Based Access Control (RBAC) mechanisms. *Need-to-know Principle*: Users are granted access strictly to the datasets and functionalities necessary for fulfilling their role-specific tasks. This granular segmentation limits the blast radius in the event of an internal or external security breach.
4. **Proactive Monitoring and Anomaly Detection.** Modern security evolves from a reactive to a proactive posture through continuous surveillance. *Auditing and Traceability*: Maintenance of immutable event logs (audit trails) that document every interaction, modification, or database query. This traceability is essential both for post-incident investigations and for compliance with international standards. *Detection Systems (IDS/IPS)*: Traffic monitoring for the identification of atypical behavioral patterns that could indicate a data exfiltration attempt or a zero-day attack.
5. **Business Continuity and Disaster Recovery.** The resilience of a CRM system is measured by its capacity to restore operations following critical incidents. *Automated Backup Strategies*: Implementation of data backup procedures with geographic redundancy, ensuring information integrity against hardware failures or ransomware attacks. *Disaster Recovery Plan (DRP)*: The existence of clear and periodically tested protocols for rapid service restoration, minimizing RTO (Recovery Time Objective) and RPO (Recovery Point Objective) indicators.

The integrated implementation of these technical and procedural measures transforms the CRM system into a trusted platform, capable of protecting the organization's informational capital and ensuring compliance with current legislative requirements.

2.8 The Impact of Cloud Computing and Containerization

The widespread adoption of cloud computing technologies has generated a fundamental shift in the lifecycle of CRM systems, redefining the processes of development, deployment, and maintenance. The transition from traditional on-premise infrastructures to cloud-native architectures has enabled organizations to benefit from unprecedented levels of agility, transforming the CRM from a static resource into a dynamic and resilient ecosystem.

Elastic Scalability and High Availability Modern CRM systems leverage the advantages of cloud infrastructure to ensure automatic horizontal scalability (auto-scaling). This capability allows the system to adjust its computational resources in real-time, based on data volume fluctuations and user interaction intensity, thereby guaranteeing consistent performance even during peak periods. Furthermore, through geographic data distribution and the

use of Availability Zones, these platforms achieve High Availability (HA) and fault tolerance standards that would have been prohibitively expensive in traditional hosting models.

Containerization and Service Orchestration An essential technological pillar in the development of contemporary CRMs is the use of containerization, primarily through Docker technology.

Portability and Isolation: Packaging the CRM application and its dependencies into isolated containers guarantees execution consistency regardless of the infrastructure environment, eliminating “it works on my machine” type conflicts.

Orchestration through Kubernetes: The use of orchestration platforms such as Kubernetes (K8s) enables automated management of the entire lifecycle of CRM microservices. This facilitates the implementation of modern deployment strategies such as Blue-Green or Canary releases, ensuring continuous updates without service interruption (zero-downtime) and efficient cluster resource management.

The Hegemony of the Software-as-a-Service (SaaS) Model The Software-as-a-Service (SaaS) delivery model remains the de facto standard in the CRM market, consolidating its position through the radical optimization of the cost-to-performance ratio.

Financial Efficiency: By transitioning from capital investments (CAPEX) to operational expenditures (OPEX), companies can access enterprise functionalities without bearing the costs of acquiring and maintaining physical servers.

Continuous Updates: The SaaS architecture enables vendors to deliver security updates and functional innovations transparently and instantaneously, ensuring that all users benefit from the latest analytics and automation technologies without complex manual interventions.

The integration of containerization and cloud services transforms the CRM system into a modular architecture capable of evolving alongside business complexity, providing a superior technological foundation for implementing artificial intelligence algorithms and large-scale data processing.

2.9 Technical Challenges

Despite offering undeniable competitive advantages, contemporary CRM systems navigate a technological landscape marked by structural constraints and the need for continuous adaptation. The complexity of these platforms grows exponentially with the diversification of digital ecosystems, generating a series of critical challenges for system architects and data managers.

Modern systems must resolve the tension between operational stability and technological dynamism:

- **Interoperability and Heterogeneous Integration:** The pressure to connect the CRM with a multitude of external platforms (ERP, social media, e-commerce solutions, BI tools) necessitates the development of robust APIs and the use of middleware capable of managing bidirectional data flows in real-time.
- **Management of Massive Data Volumes (Big Data):** The capacity to process and store immense volumes of data while maintaining low query latency requires the use of advanced storage technologies (sharding, distributed databases) and efficient data cleansing mechanisms.

- **Scalability Under Load:** Ensuring consistent performance under conditions of accelerated growth in the number of concurrent users and the complexity of automated processes represents a continuous test for infrastructure resilience.

Looking toward the technological horizon of the next decade, several emerging technologies promise to fundamentally redefine the interaction between organization and client:

- **Conversational and Generative Artificial Intelligence:** The evolution from rule-based chatbots to virtual agents grounded in Large Language Models (LLMs) will enable hyper-naturalized interactions, capable of resolving nuanced requests and generating personalized content in real-time, eliminating communication barriers.
- **Immersive Experiences through Augmented Reality (AR):** The integration of AR into sales modules will allow clients to visualize products in their own physical environment before purchase, transforming the sales process from a static presentation into an immersive sensory experience, with a direct impact on conversion rates in B2B and retail.
- **Blockchain for Data Integrity:** Distributed ledger technology can be utilized to create an immutable system for managing client identity and consent. The use of Smart Contracts within the CRM can automate the execution of Service Level Agreements (SLAs), guaranteeing total transparency and enhanced transaction security.
- **Edge Computing for Local Processing:** In the context of IoT device proliferation, the adoption of Edge Computing solutions will enable data processing at the source (at the network edge). This approach drastically reduces latency and improves privacy protection, allowing consumer behavior analysis without transferring massive volumes of sensitive data to the central cloud.

The future of CRM systems lies in their capacity to fuse these technologies into a unified architecture, transitioning from the role of a data management tool to that of an autonomous experience orchestration engine, capable of anticipating and responding to human needs with near-instantaneous precision.

Part II

Architecture and Implementation

Chapter 3

Introduction: What Is the CRM Training IT Project?

The CRM Training IT project is a Java application developed for a company that sells IT training courses. Its purpose is to centralize information about individuals interested in courses, companies that may purchase corporate training programs, sales activities, B2B opportunities, courses, training sessions, and enrollments. From a software perspective, the project is more than a simple CRUD application, even though it includes many create, search and update operations. It was designed as a software architecture demonstration in which a business problem is divided into clear layers and components that apply well-known design patterns.

The application starts from the `crm.CrmApplication` class and runs in a console environment. The `ConsoleUI` interface presents a menu with operations such as adding an individual lead, adding a corporate client, searching for a contact by email, listing hot leads, changing a lead's status, adding an activity, creating a B2B opportunity, and viewing the active sales pipeline. This console interface is not the main focus of the project but rather a simple way to demonstrate the application's functionality. The important aspect is the internal structure: how a menu command reaches the service layer, how it is validated, how scores are calculated, how data is stored, and how secondary events are triggered.

The selected domain is an excellent fit for the deployment of a CRM system, as it inherently includes two types of customers. The first category is B2C: people who are actually looking for training courses (e.g. Java Fundamentals or Python Fundamentals). The second category is B2B: companies that need training for their employees. Instead of building two separate presentation models, the author has chosen a common entity (`Contact`) that can be either an individual or a corporate customer, represented by the `contactType` field. This simplifies many workflows, since a contact can have an email address, phone number, status, score and activities regardless of its type.

The project should be treated as a CRM core. It has no web interface, almost no real authentication and only a graphical shell, but it provides a reasonably structured codebase that can be extended. If the business were ever evolved into a production application, the logic contained in the services, repositories, DAOs and domain models would not require changes: these components could act as a REST API on top of which a web interface is managed. For this reason, this part focuses mostly on describing how the code works and how the different components connect to one another.

Chapter 4

Overview of Technology Stack and Project Structure

4.1 Technology Stack

The project uses Maven, with its `pom.xml` file specifying the dependencies of the project as well as the procedure for creating an executable JAR file. The target language is Java 8, which explains the use of `Optional`, `LocalDate`, `LocalDateTime` and functional interfaces. Lombok is used to avoid repetitive boilerplate code such as getters and setters. The following annotations are used:

- `@Getter`, `@Setter`
- `@NoArgsConstructor`, `@AllArgsConstructor`
- `@Builder`, `@ToString`

This means accessor methods do not need to be implemented manually in model classes, as they are auto-generated at compile time.

4.2 Logging and Persistence Infrastructure

SLF4J together with Logback is selected as the logging framework. The Logback configuration sends log messages both to the console and to a `logs/crm-application.log` file. This is a significant choice because the project is layered and event-driven, so logs give insight into what actually ran. Existing logs show the application opening a connection pool, registering observers, performing commands and persisting contacts.

All data access uses raw JDBC; neither Hibernate nor JPA is involved. Connections are not opened and closed on each interaction. Instead, connections are pooled and managed by HikariCP, a high-performance connection pool. The `DatabaseConnection` class builds a `HikariDataSource` based on properties loaded from `AppConfig`. By default it connects to a MariaDB database on `localhost`, using `crm_training` as the database, `root` as the user and an empty password. This is acceptable for a local demonstration, but in a production-like environment such settings should be externalized into an encrypted configuration file.

4.3 Package Structure

The package structure closely maps to the architectural positions of its components:

- `crm` — application entry point (`CrmApplication`).
- `crm.ui` — the console interface (`ConsoleUI`).

- `crm.facade` — `CrmFacade`, the single entry point to the system.
- `crm.service` — business logic grouped by domain area (contacts, courses, enrollments, opportunities, notifications).
- `crm.repository` and `crm.dao` — data access.
- `crm.model.entity` (and enums) — the domain model.
- Pattern-specific packages for Command, Observer, Strategy, Builder, Factory and validation logic.

The organization is well structured and demonstrates how design patterns can be applied in practice across many different areas of a system.

Chapter 5

Application Startup and System Initialization

5.1 Configuration and Database Initialization

The startup process occurs in the `crm.CrmApplication` class, whose `main` method executes all the steps that prepare the system for execution. First, `AppConfig` is loaded. This class implements the Singleton pattern and reads application properties from an `application.properties` file when it exists, while also allowing default values for development environments. In the analyzed scenario, no properties file is present; its absence does not prevent execution but simply substitutes a default configuration.

`DatabaseConnection` is initialized after the configuration is loaded. This class also uses the Singleton pattern, but applies double-checked locking to create a single instance in a thread-safe manner. It builds a `HikariConfig` and sets the JDBC URL, username, password, MariaDB driver and connection-pool parameters, finally creating a `HikariDataSource`. Afterwards, any DAO can obtain a connection via `DatabaseConnection.getInstance().getConnection()`. The returned connection is borrowed from the pool and automatically returned to it when closed.

5.2 Event Registration and Shutdown

The `EventBus` is one of the significant components of the application, and registering observers is an essential startup task. `AuditLogObserver` is registered globally so that it receives all events. Then `WelcomeEmailObserver` subscribes to `CONTACT_CREATED`, `EnrollmentConfirmationObserver` to `ENROLLMENT_CREATED`, and `LeadScoreUpdateObserver` to `ACTIVITY_COMPLETED`. This phase shows the author differentiating between a primary action and a secondary reaction: when a contact is created, the creating code does not need to know how a welcome email is sent; it simply publishes an event and the appropriate observer reacts.

`CrmApplication` finally registers a shutdown hook — a thread executed when the JVM shuts down — to release resources such as the database connection pool. It then initializes a `ConsoleUI` and calls its `run()` method, placing the application in a loop that displays the menu, reads the user selection, and passes the corresponding operation to the facade layer.

Chapter 6

Layered Architecture

6.1 User Interface and Facade Layers

The application takes advantage of a layered architecture. At the top sits the console-based user interface, whose only responsibility is communicating with the user. Complex business logic is deliberately kept out of `ConsoleUI`: it reads input, creates objects using builders, and calls `CrmFacade` methods. Because the interface may change over time, it is kept separate. If a REST API were created later, it could reuse the same `CrmFacade` or services without changing any business logic.

The facade wraps all complexity and provides a single entry point instead of forcing the UI to know about every service, command and repository. Methods such as `createContact`, `getHotLeads`, `changeLeadStatus`, `createOpportunity` and `logActivity` delegate to the corresponding services and commands. This is the Facade pattern: a single point of access for all subsystems.

6.2 Service, Repository and DAO Layers

Business logic lives in the services underneath the facade. The actual rules of the application — validation, duplicate removal, score calculation, status updates, price calculation, event publishing and default-value assignment — are encapsulated in these services. For example, `ContactService` does much more than save a contact: it validates data, checks for email presence, calculates a score using a strategy, saves the contact, and publishes an event. This is where the boundary between DAO and service lies: the DAO operates at the SQL level, while the service operates on business rules.

Below the services are repositories and DAOs. A repository (such as `ContactRepository`) exposes domain-oriented methods — for example `findHotLeads`, `findByEmail` and `findByLeadStatus`. The DAO holds the real operations: SQL `INSERT`, `UPDATE` and `SELECT` statements that map `ResultSet` objects into Java entities, with parameters bound inside `PreparedStatement` objects.

6.3 Simplified Flow

The overall request flow can be summarized as follows:

```
ConsoleUI → CrmFacade → CommandInvoker → Service
Service → Repository → DAO → Database
Service → EventBus → Observers
Service → Strategy / Validator / Factory / Builder
```

Chapter 7

Domain Model and Main Entities

7.1 Base Entity

The domain model resides in the `crm.model.entity` package. `BaseEntity` is a base class that abstracts the attributes common to all entities: `id`, `createdAt` and `updatedAt`. `BaseEntity` implements the `isNew()` method to determine whether a given object should be inserted or updated from the repository's point of view. If `id` is `null`, the object is new and `insert` is called; otherwise `update` is called.

7.2 Contact

The `Contact` is the central entity of the system. It includes customer-related fields such as `firstName`, `lastName` and `birthDate`, as well as company-related fields such as `companyName`, `fiscalCode`, `registrationNumber`, `industry` and `employeeCount`. The shared fields are `email`, `phone`, `leadSource`, `leadStatus`, `leadScore`, `experienceLevel`, `learningGoal`, `assignedTo` and `gdprConsent`. For corporate contacts, `getFullName()` returns the company name; for individuals, it returns the first and last name. Domain methods such as `isCorporate()` and `isIndividual()` add small but meaningful value, making the code more readable and expressive.

7.3 Course and CourseSession

`Course` represents the educational offering. It has a `code`, `name`, `description`, `syllabus`, `category`, `level`, `prerequisites`, `duration`, `individual price`, `group price`, `corporate daily price`, `participant limits`, and the number of participants it supports. `CourseSession` is a specific scheduled occurrence of a course, with properties such as `start date`, `end date`, the trainer running the course, the delivery method, the training location, the maximum number of participants and the status. The distinction matters: a course is the generic educational product, whereas a session is a specific occurrence scheduled in time.

7.4 Enrollment, Opportunity and Activity

An `Enrollment` indicates that a contact has enrolled in a course session. It holds `contactId`, `sessionId`, `enrolledDate`, `status`, `price`, `discount`, the final amount, the total amount paid, the paid/unpaid flag, `attendance`, `remarks`, certificate information and any prior grade. The `getRemainingAmount()` method computes how much the participant still has to pay, an example of domain logic that depends only on the object's own state.

`Opportunity` models the B2B sales pipeline. It consists of a `title`, `description`, `estimated number of participants`, `customer-specific requirements`, `delivery mode`, `estimated value`, `pro-`

positional value, probability, stage and expected closing date, and belongs to a corporate client. The methods `isClosed()`, `isWon()` and `getWeightedValue()` provide insight into an opportunity's status; `getWeightedValue()` returns the proposed value weighted by probability.

Activity records interactions with a contact or an opportunity — email, phone calls, meetings, tasks, notes and demos. It carries a subject, description, scheduled date-time, completion date-time, duration, status, priority, outcome and next steps. The methods `isCompleted()` and `isOverdue()` are useful for tracking whether an activity has been completed and whether it is overdue.

Chapter 8

Contacts, Leads and Score Calculation

8.1 Contact Creation Flow

Contact creation usually starts from `ConsoleUI`. For an individual lead, the UI reads first name, last name, email and phone and creates a `ContactBuilder`. If the contact is an individual, the builder sets `contactType`, name, email, phone, lead source, experience level, learning goal and GDPR consent. For a corporate client, the same builder sets `contactType` to corporate along with the company name, fiscal code, industry and number of employees. This allows fluent, ergonomic construction code.

After construction, the lead goes to `CrmFacade.createContact`. The facade does not save the contact directly; instead it creates a `CreateContactCommand` and passes it to `CommandInvoker`. The invoker executes the command and stores it in a history of 100 slots. At present this history is primarily a proof of concept, but it can later support audit logging, execution queues or undo functionality.

8.2 Validation and Uniqueness

`CreateContactCommand` invokes `ContactService.createContact`, where the real business logic happens. The process begins with `ContactValidatorChain`, which consists of `EmailValidator`, `PhoneValidator`, `ContactTypeValidator` and `GdprValidator`. `EmailValidator` checks the format and presence of the email. `PhoneValidator` only checks the number when one is provided. `ContactTypeValidator` ensures an individual contact has a first or last name, while a corporate contact has a company name. `GdprValidator` checks the consent date when `gdprConsent` is true.

`ContactService` then verifies whether the email address is already registered. The uniqueness rule is important, since the `contacts` table must contain unique emails; a `BusinessException` is thrown if the email already exists. If everything is valid, the service assigns a new status and the first/last contact dates.

8.3 Lead Scoring Strategies

The `SimpleLeadScoringStrategy` calculates the initial score based on data completeness and the lead source. The email, phone, name, source, experience level, learning goal, GDPR consent and marketing permissions all feed into the score. A referral lead is worth more than a cold call, and goals such as finding a new career earn extra points. `B2BLeadScoringStrategy` considers company factors such as fiscal data, sector, number of employees and location; companies with more employees or in IT/software verticals score higher.

Once a score is determined, it is saved via `ContactRepository` and `ContactDao`. Finally, `ContactService` publishes a `ContactCreatedEvent` via the `EventBus`. This triggers two reac-

tions: **AuditLogObserver** records the event to the **audit_logs** table, and **WelcomeEmailObserver** sends a welcome email through **NotificationService**. The email is not actually sent via SMTP but simply logged; nevertheless the architecture shows how a real notification service could later be incorporated.

Chapter 9

EventBus, Activities and Automated Reactions

9.1 Activity Creation

Activities represent interactions between the sales team and contacts. Through `ConsoleUI`, the user can create emails, phone calls, meetings and notes. These are created through `ActivityFactory`, which hides the creation details and assigns default values where appropriate. A phone call lasts about half an hour and has high priority, an email has medium importance, and a note is automatically marked as completed. This avoids defining default values repeatedly in the UI and other components.

When an activity is created, `CrmFacade` calls `ActivityService.createActivity`. If status and priority are not supplied, the service defaults to scheduled status and medium priority, then saves through `ActivityRepository`. A newly scheduled activity does not necessarily represent genuine interest, so no score event is published at that time. The score should only change when the activity is actually performed or a definitive result is available.

9.2 Activity Completion and Score Updates

`ActivityService.completeActivity` handles completion: it fetches the activity by id, sets its status to completed, assigns the current timestamp to `completedDate`, stores the outcome and next steps, and saves the activity. After persistence, the service publishes an `ActivityCompletedEvent`. This event is consumed by `LeadScoreUpdateObserver`, which queries the lead's activity and converts completed activities into lead-score changes.

The observer verifies that the event is an `ActivityCompletedEvent`, inspects the activity, checks for a `contactId`, and maps each activity type to a scoring code — for example `EMAIL` → `EMAIL_OPENED`, `CALL` → `CALLBACK_REQUESTED`, `MEETING` → `MEETING_ATTENDED`, `DEMO` → `DEMO_REQUESTED`. It then invokes `ContactService.updateLeadScoreFromActivity`, which recalculates the effect with the corresponding strategy and keeps the lead score within the 0–100 range.

9.3 Benefits of the Event-Driven Design

This is a clean small-scale example of event-driven design. The activity service is not aware of the scoring rules and is therefore decoupled from the scoring strategy; its only duty is to declare that an activity has been completed. A specialized observer absorbs the side effects. This allows future functionality — such as sending notifications after meetings, automatically creating follow-up tasks, or updating dashboards — to be implemented by adding new observers rather than modifying `ActivityService`.

Chapter 10

B2B Opportunities and the Sales Pipeline

10.1 Opportunity Modeling

Opportunities model the sales process for corporate clients. An opportunity refers to a specific business conversation — for example, discussing Java training for a software development team. The `Opportunity` entity holds `clientId`, title, description, estimated number of participants, custom requirements, delivery mode, preferred location, estimated start date, estimated value, demand value, proposal value, stage, probability and expected closing date.

Opportunities are created with `OpportunityBuilder` from the UI. If the stage is not specified explicitly, the builder sets it to `LEAD_QUALIFICATION` together with the probability corresponding to that stage. This ensures the `Opportunity` object is in a valid state even when the user does not populate every field.

10.2 Pipeline Stages

The pipeline is controlled by the `OpportunityStage` enumeration: `LEAD_QUALIFICATION`, `NEEDS_ANALYSIS`, `PROPOSAL_SENT`, `NEGOTIATION`, `CONTRACT_REVIEW`, `WON` and `LOST`. Each stage has a corresponding default probability; for example, `LEAD_QUALIFICATION` carries a lower probability and `CONTRACT_REVIEW` a higher one, with `WON` at 100% and `LOST` at 0%. When an opportunity transitions from one stage to the next, the system re-calibrates the probability accordingly.

10.3 Opportunity Services and Events

`CrmFacade.createOpportunity` delegates execution through `CreateOpportunityCommand` to `OpportunityService.createOpportunity`. The service checks that `clientId` and title are defined, applies default values for stage and probability when not explicitly provided, and saves the opportunity. `MoveOpportunityStageCommand` invokes `OpportunityService.moveToStage` to move an opportunity through the pipeline.

After a stage transition, `OpportunityService` publishes an `OpportunityStageChangedEvent`. There is currently no dedicated observer for this event, so the globally registered `AuditLogObserver` handles it, ensuring all stage changes are tracked. The architecture is extensible: behavior could later be added to notify a manager when an opportunity enters `NEGOTIATION`, to generate a proposal on entering `PROPOSAL_SENT`, or to create a contract on entering `WON`.

Chapter 11

Courses, Sessions, Enrollments and Pricing Strategies

11.1 Educational Entities

Courses, sessions and enrollments represent the educational side of the CRM. A **Course** is a product, defined by code, name, description, duration, category, level and pricing information. A **CourseSession** is one delivered instance of a course, with a trainer, delivery method, duration and capacity. An **Enrollment** links a contact and a session, recording pricing, discounts, payments, attendance, grades and certificates.

11.2 Course Service

CourseService holds the business logic for courses. When creating a course it validates that the course code is unique, that the name is provided and that the duration is positive. It assigns defaults — `active = true`, `minParticipants = 3` and `maxParticipants = 15` — where values are missing, and saves the course through **CourseRepository**. This illustrates how the service layer works: business rules are enforced by the service, not by the DAO.

11.3 Enrollment Service and Pricing Strategy

EnrollmentService uses the Strategy pattern to calculate pricing. Its `enrollContact` method receives a `contactId`, a `sessionId`, a **Course** object and a pricing strategy. If no strategy is set, it defaults to **IndividualPricingStrategy** without an early-bird discount. The service also verifies that the contact is not already enrolled in the session before creating the enrollment, enforcing the unique constraint defined in the SQL schema (no user can be enrolled twice in the same session).

Chapter 12

Persistence: Repository, DAO and JDBC

12.1 Repository and DAO Separation

Data access is organized into two layers: Repository and DAO. The repository layer is aligned with domain language, while the DAO layer is aligned with SQL. A repository such as `ContactRepository` exposes methods like `findByEmail`, `findByLeadStatus`, `findHotLeads`, `findByAssignedTo`, `findStaleLeads` and `countByLeadStatus`; these bind to the actual SQL statements in `ContactDao`. With this separation, the services calling repositories never have to deal with `PreparedStatement` objects.

12.2 AbstractRepository and AbstractDao

`AbstractRepository` contains generic operations such as `save`, `findById`, `getId`, `findAll`, `count` and `deleteById`. The `save` method checks `entity.isNew()`: if the id is null it calls `DAO.save`, otherwise `DAO.update`. This is good design, since services do not need to track whether an object is new or existing. The layer also introduces `ResourceNotFoundException`, thrown from `getId` when the requested entity is not found.

`AbstractDao` is one of the most technical classes in the project and is an implementation of the Template Method pattern. The abstract class defines the high-level algorithm for operations such as `save`, `update`, `findById`, `findAll`, `count`, `deleteById` and `existsById`, while allowing subclasses to define details: the table name, the insert and update SQL, the result-set mapping, the parameter binding, and generated-key handling. It consolidates the repetitive JDBC code common across repositories and leaves the per-entity variation to concrete DAOs.

12.3 Concrete DAOs

The most complex DAO is `ContactDao`. It builds an `INSERT` query with the relevant `contacts` columns, orders parameters correctly, converts enums to strings and local dates/timestamps to SQL types. For retrieval, the `mapResultSetToEntity` method rebuilds a `Contact` from table columns; for enums it uses `parseEnum`, which returns null when the stored value does not correspond to a valid enumeration constant. This prevents crashes on invalid data but can also hide it.

`CourseDao`, `EnrollmentDao`, `OpportunityDao` and `ActivityDao` follow the same scheme, differing only in their SQL statements, mapping and entity-specific queries. `OpportunityDao` includes `findByClientId`, `findByStage` and `findActivePipeline`; `ActivityDao` includes `findByContactId`, `findByOpportunityId` and `findUpcoming`. These mirror the real questions the CRM must answer: which activities relate to a contact, which opportunities are still active, which courses exist, and which participants have failed payment.

12.4 Transactions and Partial Failures

The project relies on `PreparedStatement` so that parameters are bound rather than concatenated into SQL strings (except for application-managed table names), which greatly reduces the risk of SQL injection from user input.

However, not every compound operation is transactional. Saving a contact and publishing events — after which observers write to `audit_logs` — is not wrapped in a single transaction. If a non-primary operation fails, the error is logged but the whole workflow is not rolled back, so partial failures are possible.

Chapter 13

Design Patterns and Their Role in the Project

13.1 Creational and Structural Patterns

This project was specifically created to showcase design patterns. The Singleton pattern is the most commonly used: `AppConfig` provides a single source of configuration, `DatabaseConnection` a single connection pool, and `EventBus` a single event channel, while services, repositories and DAOs are usually obtained through `getInstance()` methods. This brings obvious simplicity to a small console application, at the cost of harder testing, since global instances cannot easily be replaced by mocks.

`CrmFacade` represents the Facade pattern, abstracting the complexity of the underlying subsystems. The UI does not need to know whether it interacts with a command, service, repository or observer; `createContact` is simply a safe way to create a contact, while validation, scoring, persistence and event publishing are triggered internally.

The Builder pattern appears in `ContactBuilder` and `OpportunityBuilder`. Although Lombok could generate generic builders, the author wrote custom ones to obtain expressive methods such as `asIndividual`, `asCorporate`, `withGdprConsent`, `participants` and `expectedClose`, with default values and minimal validation before construction.

13.2 Behavioral Patterns

The Command pattern manifests through the `Command` interface, `AbstractCommand`, `CommandInvoker` and concrete commands such as `CreateContactCommand`, `UpdateLeadStatusCommand`, `EnrollContactCommand`, `CreateOpportunityCommand` and `MoveOpportunityStageCommand`. `AbstractCommand` handles logging and execution-time measurement via the Template Method pattern: `execute` is final and subclasses implement `doExecute`. `CommandInvoker` keeps command history, which can later support auditing or undo.

The Observer pattern is implemented through `EventBus`, `CrmEvent` and the observer classes. Services publish events, and observers react — for example, `ContactCreatedEvent` triggers a welcome email, `EnrollmentCreatedEvent` confirms enrollment, `ActivityCompletedEvent` updates the lead score, and `AuditLogObserver` receives all events. This decouples the main action from its side effects.

The Strategy pattern is used for lead scoring and pricing. `LeadScoringStrategy` has separate B2C and B2B implementations, and `PricingStrategy` has individual and corporate implementations, so algorithms can change without modifying the services that use them. If, for example, the company wanted a scoring strategy based on website scrolling behavior, it would simply add a new class implementing `LeadScoringStrategy`.

The Factory pattern is used for activities and notifications, the Chain of Responsibility

pattern for contact validation, and the Template Method pattern in both `AbstractDao` and `AbstractCommand`.

13.3 Summary of the Design Patterns

At the highest level, the project uses design patterns to separate intent from implementation: Facade hides complexity, Command turns actions into objects, Observer decouples side effects, Strategy makes algorithms interchangeable, and the Repository/DAO split separates domain language from SQL implementation. Each pattern fulfills a clear role in the application workflow.

Chapter 14

Validation, Exceptions and Logging

14.1 Validation Chain

Contacts are validated more strictly than other entities. `ContactValidatorChain` forms a chain of validators: `EmailValidator`, `PhoneValidator`, `ContactTypeValidator` and `GdprValidator`. Each validator inspects the entity and a list of validation errors, queues its own errors, and passes processing to the next validator. Finally, `ContactValidatorChain` returns the validation errors or throws a `ValidationException`. This structure makes it easy to add new validators — for example, one that validates a company fiscal code.

14.2 Exception Hierarchy

The custom exception hierarchy is straightforward but beneficial. `CrmException` is the base runtime exception, representing business problems such as a duplicate email or missing required fields. `DataAccessException` signals a database- or SQL-related issue. `ResourceNotFoundException` is thrown when an entity is searched by id and does not exist. `ValidationException` carries a list of validation errors and composes its message from them. This hierarchy makes the code more expressive than relying on a generic `RuntimeException`.

14.3 Logging

Most key components are logged. `CrmApplication` logs startup and shutdown; `DatabaseConnection` logs pool initialization and shutdown; services log business operations such as contact creation, course creation, enrollment, payment registration and opportunity-stage transitions; `CommandInvoker` and `AbstractCommand` record command execution; `EventBus` logs observer registration and event notification; and DAOs log SQL exceptions and save/update operations at debug level.

From a learning standpoint, logs are one of the best available tools, since they show the actual execution flow — command invocation, score calculation for new contacts, persistence to the `contacts` table, event publication, and simulated email delivery before the command commits. This makes the architecture visible without a debugger.

14.4 Validation Gaps

There are areas where validation could be strengthened. Opportunities currently validate only `clientId` and title. Enrollments only check for double registration in a session; they verify neither session capacity nor seat availability. Payment registration does not check that a payment amount is positive. Activities do not validate fields such as `contactId`, subject or

activity type. These gaps are acceptable for a demonstration project but are worth noting as improvements for future iterations.

Chapter 15

Database, SQL Schema and Consistency Observations

15.1 Conflicting Schema Sources

The project contains two SQL schema sources: `sql/schema.sql` and `src/main/resources/schema.sql`. The first corresponds to the DAO implementations — for example, `courses.name`, `price_individual`, `price_group`, `opportunities.client_id` and `probability_percent`, matching `CourseDao` and `OpportunityDao`. The second uses slightly different names, such as `courses.title`, `base_price`, `opportunities.contact_id` and `probability`. This difference is significant because the application can still start against the wrong schema, but database operations may then fail at runtime.

15.2 Schema Initialization

The `AppConfig` configuration includes a property `db.init.schema=true`, but the codebase does not include a component that reads the schema SQL and creates the tables automatically. The JDBC URL with `createDatabaseIfNotExist=true` creates the `crm_training` database but not its tables. Project logs confirm this: in one run the application connected to MariaDB successfully but listing leads produced the error “Table `crm_training.contacts` doesn’t exist”. The schema must therefore be run explicitly before starting the application.

15.3 Schema Richness and Indexing

The `sql/schema.sql` file is richer than what the console UI exposes. It includes tables for contacts, courses, trainers, sessions, enrollments, opportunities and activities, but also proposals, contracts, invoices, payments, lead-score logs and audit logs. This implies the author envisioned a deeper CRM tool than the current console demonstration shows; although the accompanying services are not all implemented, several tables are designed for future extension.

The schema defines indexes based on common query patterns. Email is indexed for fast contact searches; `lead_status`, `assigned_to` and `contact_type` are indexed for filtering; the `courses` table is indexed on code and category; the `opportunities` table on stage; and the `activities` table on `activity_type` and `scheduled_date`. These correspond well to common CRM needs.

15.4 Auditing

Auditing is implemented through the `audit_logs` table and `AuditLogObserver`. The observer obtains the entity id via reflection (calling `getId()` on the event payload) and populates `entity_type`, `entity_id`, `action` and `new_values`. This approach is generic and works for any event whose payload exposes an id. As implemented, however, the audit mechanism stores `payload.toString()`, which can be hard to compare or analyze automatically. A more sophisticated version would store structured JSON containing both the previous and current values.

Chapter 16

Features and Non-Functional Requirements Derived from the Code

16.1 Functional Requirements

The project has no separate requirements document; its requirements can be inferred from the menu options, services, domain models and SQL schema. The main functional requirements are to manage B2C and B2B contacts, make contacts searchable by email, calculate and update lead scores (automatically per engagement rules or manually), list hot leads, transition lead statuses, manage activities, manage opportunities along the sales pipeline with weighted closing values, give managers a pipeline view, manage active courses and sessions, register payments on course enrollments, and complete enrollments with certificate generation.

A contact is either B2C or B2B. A B2C contact must have a name, email, phone, lead source, proficiency and learning goal. A B2B contact must store the company name, fiscal code, sector and number of employees. For both types, the system keeps status, lead score, activity and GDPR-consent information. This reflects the nature of a training company that serves both individual and corporate customers within a single system.

16.2 Non-Functional Requirements

The non-functional requirements are also apparent. The multilayered architecture indicates a need for maintainability. The Repository/DAO split reflects a need to decouple business logic from SQL. The use of HikariCP indicates attention to efficient database-connection management. Logging reflects a need for observability. The Singleton pattern provides easy access to shared resources for a console application, while the observers provide the flexibility needed for secondary effects and event-driven behavior.

16.3 Security

Security is partially addressed. The schema includes a `password_hash` column for users, but authentication is not implemented in the UI. `PreparedStatement` prevents SQL injection, and GDPR-related fields and validation support compliance. However, there is no access-control mechanism or encryption, no safeguard for configuration passwords, and no session management. These are constraints of the current realization.

16.4 Scalability

Scalability is sufficient for a local or proof-of-concept application. HikariCP enables effective connection management, and database indexes improve query performance. However, there

is no distributed architecture, no caching layer, and no stateless API; the global Singleton instances would not be reliable in a test environment or multi-instance deployment. These design decisions are acceptable for a bachelor's thesis whose primary focus is a clear architecture and applied design patterns.

Chapter 17

Limitations and Possible Improvements

17.1 Key Limitations

The most important limitation is the inconsistency between the two SQL schema files. For stability, there should be one official schema or, better, a database-migration framework such as Flyway or Liquibase. This would allow the database structure to be created and updated reliably and would avoid most mismatches between DAOs and tables.

The second limitation is the lack of automated testing. Although JUnit 5 is included in `pom.xml`, the `src/test` directory contains no tests. Unit tests should cover scoring strategies, pricing strategies, validators, builders and services, and integration tests with a dedicated test database should cover the DAO classes. At present, correctness is verified mainly by running the application by hand and checking the logs.

The third limitation is the pervasive use of Singletons. Convenient for an academic project, this is problematic for testing and production: services instantiate their own repositories, repositories their own DAOs, and the database connection is used directly. Adding dependency injection — manually or via a framework such as Spring — would ease mocking and component replacement, especially during testing.

The fourth limitation is the dated user interface. `ConsoleUI` suffices for a demo, but a real CRM needs at least a REST API and a web UI. Useful features would include contact lists, contact detail pages, activity timelines, a Kanban-style sales pipeline, course-management screens and statistical dashboards. The existing business logic would not have to change when a web interface consumes the service-layer methods or controller endpoints.

The fifth limitation is the absence of dedicated transaction management in complex workflows. Enrolling a contact creates an enrollment record, updates the contact status and publishes an event; if any step fails, there is no transaction manager to roll everything back. In production, core business transactions should run inside well-defined database transactions.

17.2 Further Improvements

Other possible improvements include: real email and SMS provider integrations; invoice creation; better use of the existing proposals and contracts tables; role-based authentication and authorization; stricter payment validation (rejecting empty payments); stricter requirements for course sessions to be created with valid start/end dates within the application's timezone; report export from dashboards (per teacher, student or class group); sales metrics presented on dashboards and widgets; and resolving the Romanian-character encoding issues that arise from static file paths.

Bibliography

- [1] Francis Buttle and Stan Maklan. *Customer Relationship Management: Concepts and Technologies*. Routledge, London, UK, 4th edition, 2019.
- [2] Injazz J. Chen and Karen Popovich. Understanding customer relationship management (CRM): People, process and technology. *Business Process Management Journal*, 9(5):672–688, 2003.
- [3] Qimei Chen and Hsiang-Ming Chen. Exploring the success factors of eCRM strategies in practice. *Journal of Database Marketing & Customer Strategy Management*, 11(4):333–343, 2004.
- [4] Forrester Research. *The Forrester Wave: Sales Force Automation, Q3 2022*. Research report, Forrester Research, Cambridge, MA, 2022.
- [5] Grand View Research. *Customer Relationship Management Market Size, Share & Trends Analysis Report 2023–2030*. Market research report, Grand View Research, San Francisco, CA, 2023.
- [6] IDC. *Worldwide Customer Relationship Management Applications Software Market Shares, 2021: Market Growth Continues Despite Economic Uncertainty*. Research report, International Data Corporation, Framingham, MA, 2022.
- [7] V. Kumar and Werner Reinartz. *Customer Relationship Management: Concept, Strategy, and Tools*. Springer, Berlin, Germany, 3rd edition, 2018.
- [8] Wes McKinney. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O’Reilly Media, Sebastopol, CA, 2nd edition, 2017.
- [9] McKinsey & Company. *The Future of CRM: How Technology is Reshaping Customer Relationships*. Research report, McKinsey Global Institute, New York, NY, 2021.
- [10] John Miller. Building Scalable CRM Solutions with Django and PostgreSQL. *Python Magazine*, 15(3):45–52, 2022.
- [11] Adrian Payne. *Handbook of CRM: Achieving Excellence in Customer Management*. Butterworth-Heinemann, Oxford, UK, 2012.
- [12] Jennifer Smith and Mark Williams. Best Practices for CRM Implementation in SMEs. *Harvard Business Review*, 99(4):78–85, 2021.
- [13] Emma Taylor. The Evolution of Customer Relationship Management: From Contact Management to AI-Powered Insights. *MIT Technology Review*, 126(2):34–41, 2023.